Team Control & Navigation

From environment installation and our first manually controlled robot to tested autonomous warehouse navigation and ROS 2 visualization.

Prepared for: Helpforce AI R&D Training Program

Prepared by: Hamna Qaseem & Ahmed Marie

Project environment: NVIDIA Isaac Sim 5.1.0 | Ubuntu | Python | VS Code | MobilityGen exploration | ROS 2/Nav2 exploration

Submission: 19 October 2026

Repository: https://github.com/Hamna12/warehouse-control-navigation

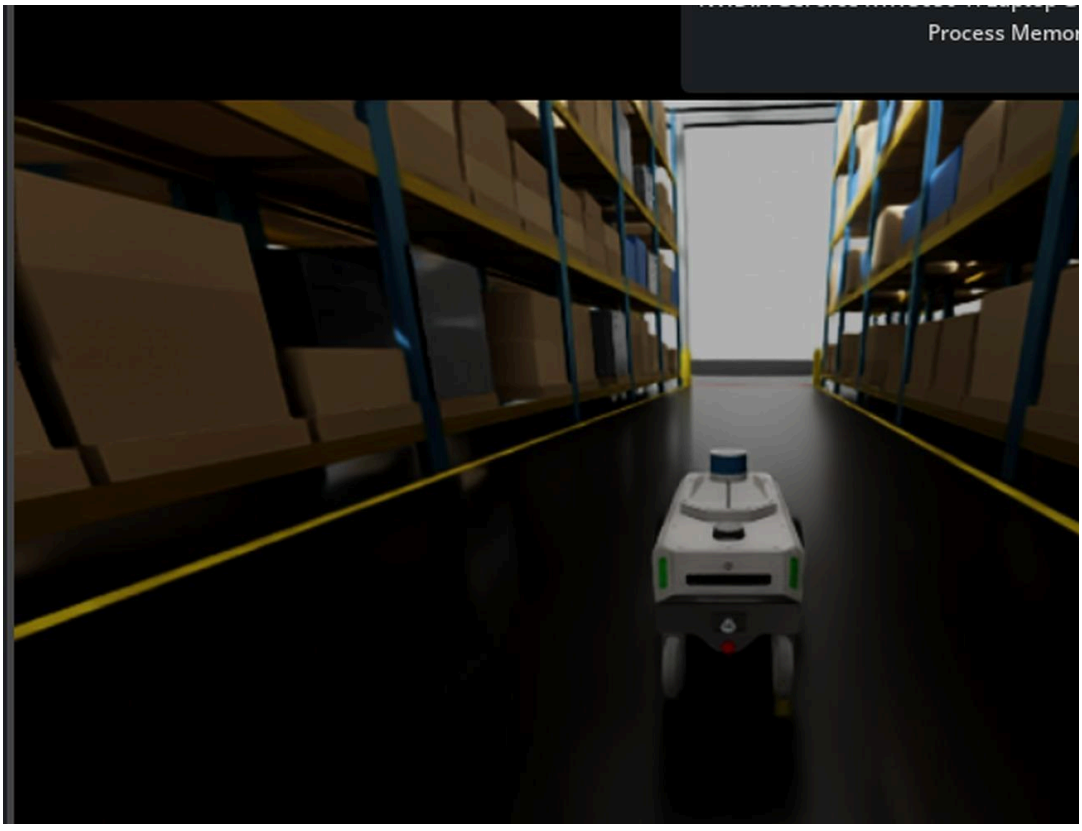


Figure — Final project overview: Carter robot operating in the warehouse.

Executive Summary:

During the Helpforce AI R&D 8-week Training Program, we developed the Control & Navigation component of a simulated warehouse robotics project. Our work began before robot programming: we first had to make NVIDIA Isaac Sim 5.1.0 run on limited laptop hardware. We then built our own four-wheeled robot, implemented manual W/A/S/D control, placed it in the warehouse, explored camera and LiDAR visualization, and developed Python-based tools. We progressed to a custom autonomous controller and static-map A* route planning for our original robot, and subsequently moved to a Carter-based robot setup while exploring NVIDIA MobilityGen. We attempted ROS 2 Nav2 integration, encountered performance and synchronization issues, and implemented a working in-simulator Python navigation interface. Finally, we launched NVIDIA's official ROS 2 warehouse navigation sample with RViz2 map, LiDAR, and camera visualization. These are separate milestones, and this report states the evidence and limitations of each.

Reporting Method and Timeline:

This report is organized as eight chronological development weeks/phases to meet the requested week-by-week format.

1. Environment Setup and First Manually Controlled Robot

1.1 Initial objective

Our first objective was to install Isaac Sim, create a basic mobile robot and verify that it could move under direct operator control. Before any navigation experiment, we needed a stable simulation environment, functioning wheel joints and repeatable keyboard commands.

1.2 Problem 1: Insufficient storage

Our first installation attempt on the laptop's internal drive could not be completed because of insufficient free disk space. The installation guidance and our planning assumed at least 50 GB of available storage. We added an external SSD and moved the Isaac Sim installation to that drive. This allowed installation to complete, although it did not solve the later rendering startup problem.

1.3 Problem 2: GPU VRAM compatibility

After installation, Isaac Sim crashed when launched. We ran the compatibility checker and compared our laptop against the values reported in that check. The checker indicated that video memory was the failing category; other listed components passed.

Component	Checker requirement	Our machine	Result
GPU VRAM	10 GB minimum	4.29 GB	Failed
GPU	RTX-series GPU	RTX 3050 Ti laptop	Passed
NVIDIA driver	535 or newer	595.91.7	Passed
System RAM	32 GB	33.36 GB	Passed
Storage	50 GB	432 GB	Passed
Operating system	Ubuntu 22/24	Ubuntu 22.04	Passed

The requirements above reproduce our reported compatibility-check values; they should not be interpreted as a universal specification for every Isaac Sim configuration.

1.4 Problem 3: Crash before the interface loaded

The application initially showed a black screen and terminated with a segmentation fault before the user interface became available. The terminal trace pointed to librtx.scenedb during RTX renderer initialization, consistent with VRAM pressure. We tried portable mode, clearing shader caches, several rendering-related launch flags, and disabling costly effects such as shadows, reflections and ambient occlusion. These attempts did not immediately resolve the failure because the renderer could stop before we could change settings in the UI.

1.5 Subsequent startup progress

We continued troubleshooting startup, driver and rendering conditions until we could open Isaac Sim and load a scene. Our first progress presentation records a working scene and robot motion, while also noting startup freezes and slow scene loading. We do not attribute the eventual success to one specific undocumented fix. The limited GPU remained a performance constraint throughout the program.

1.6 Building our original robot

Once the simulator was usable, we created an initial four-wheeled robot (referred to as IsaacBot in the later navigation work). We configured its body, wheels, wheel joints, physical behavior and movement graph. We then implemented keyboard control: W to move forward, S to reverse, and A/D to turn. At this stage the operator selected the route and drove the robot manually; there was no automatic path planner.

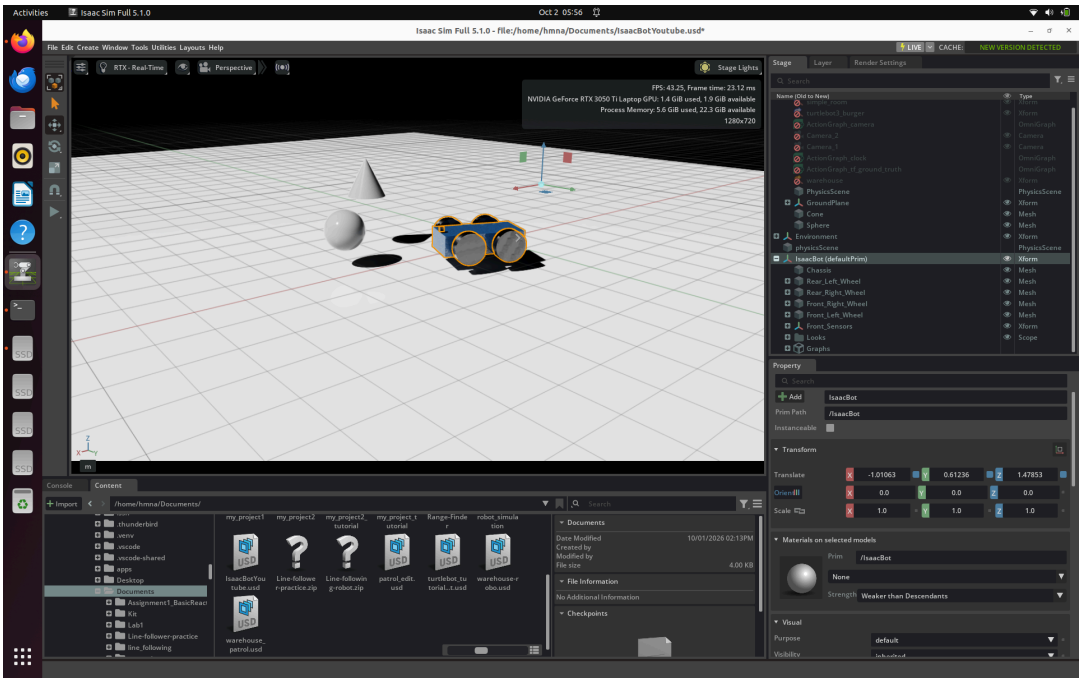


Figure — Initial manual robot control with W/A/S/D movement.

Outcome:

We had a manually driven robot inside Isaac Sim, with basic motion responding to operator input. The installation and rendering challenges had been overcome sufficiently for continued development, but autonomous navigation had not yet been implemented.

Warehouse Integration, Sensors and Python Workflow

2.1 Moving our robot into the warehouse

We loaded the supplied warehouse USD environment and integrated the original robot into it. This required checking the robot's starting position, height above the floor, orientation, wheel contact and camera framing. The scene contained storage racks and aisles, providing the physical layout for later navigation experiments. We had to distinguish simulator issues, such as robot placement and physics, from controller issues.

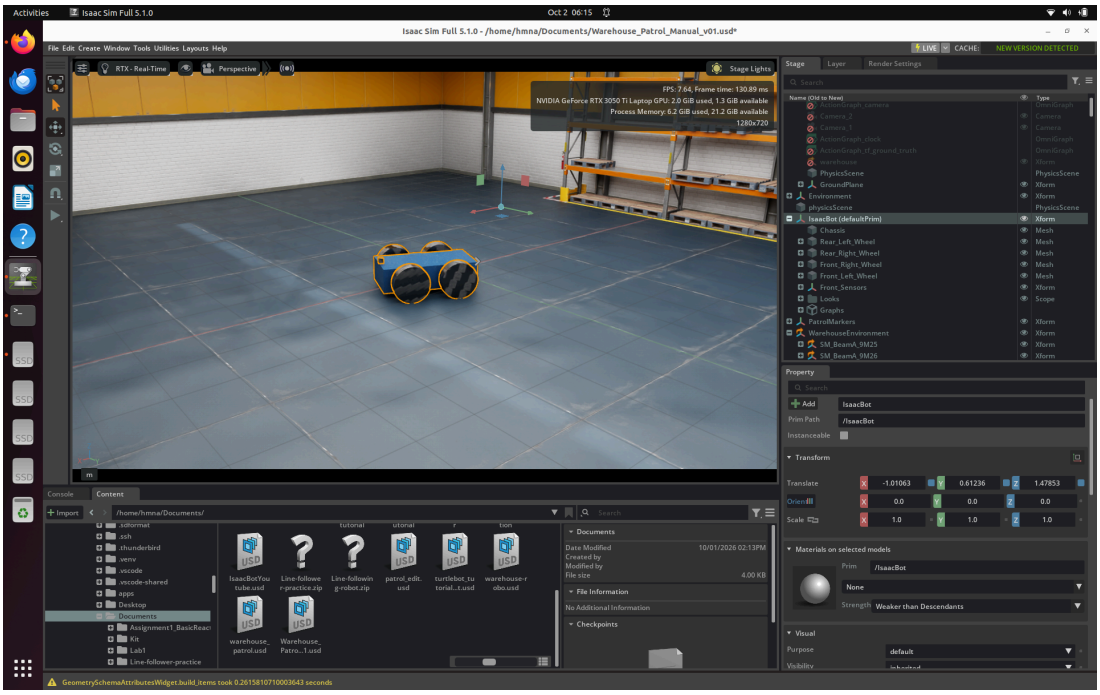


Figure — Our original robot integrated into the warehouse scene.

2.2 Camera integration and visual checks

We explored camera views to observe the robot and warehouse. We encountered black or unusable views and framing problems; adjusting camera pose and focal settings helped produce usable images. The camera was used for visualization and alignment checks. We did not establish camera-based autonomous localization or visual obstacle detection at this stage.

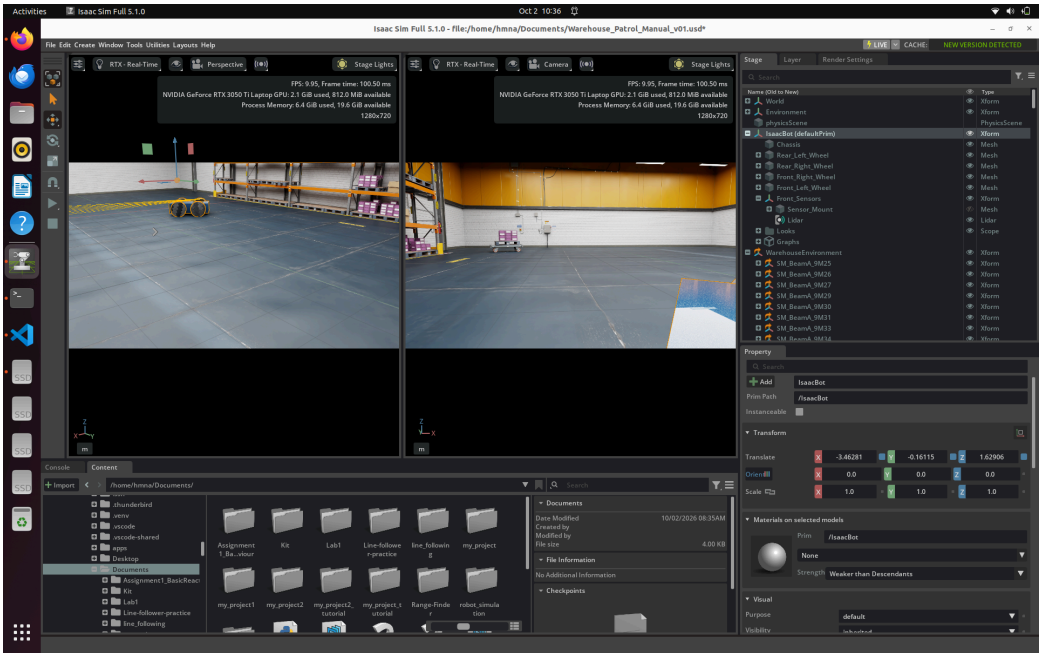


Figure — Camera-view setup and alignment checks.

2.3 LiDAR visualization

We added or inspected LiDAR range visualization, observed rays interacting with warehouse structures and investigated sensor mounting and alignment. This helped us understand sensing in simulation. However, visible LiDAR rays were not equivalent to integrating LiDAR detections into the controller; our later verified A* route planning used known static geometry or occupancy-map information.

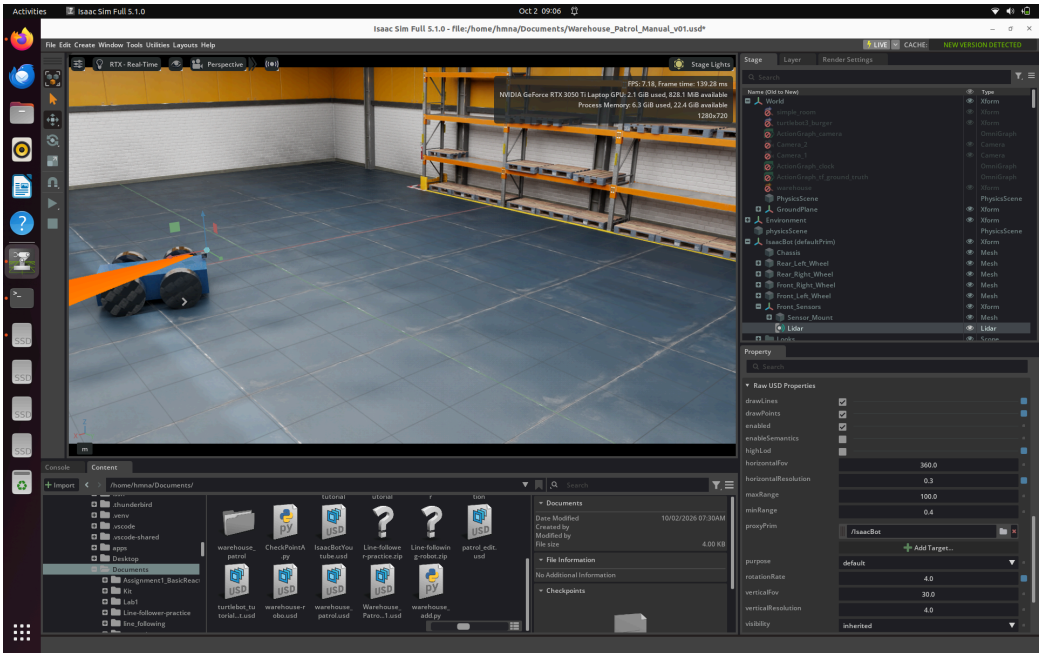


Figure — LiDAR ray visualization in the warehouse.

2.4 Connecting VS Code, Python and Isaac Sim

We developed a workflow for editing Python in VS Code and executing scripts inside the running Isaac Sim application. This separated the code-editing workflow from simulator operation and allowed us to iterate on robot behavior, dashboards and debugging without rebuilding the entire scene each time.

2.5 Manual patrol dashboard

We created an initial Python patrol panel to log operator actions: starting or finishing a patrol, reporting checkpoint arrival, and recording obstacle-related events. Entries included timestamps. These reports were manually triggered; they were not automatic proof that a waypoint was reached or that an obstacle had been detected. The dashboard was an early user-interface and event-logging milestone.

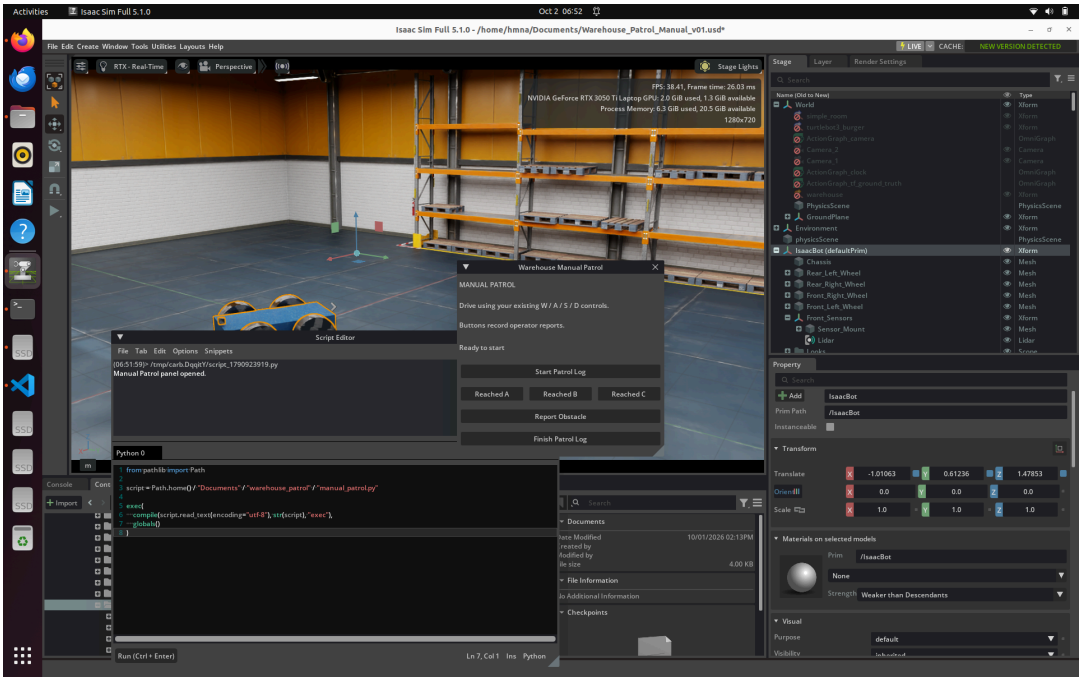


Figure — Initial Python manual patrol dashboard and event logging.

Outcome

We had the original robot working in a warehouse scene, camera and LiDAR visualization, and a Python-based operator dashboard. This phase established the environment and software workflow needed for autonomous control.

Autonomous Controller and Static-Map A* Planning

3.1 Transition from manual to autonomous movement

Our next task was to accept a destination and control the robot automatically. Early attempts exposed several issues: incomplete wheel actuation, competing keyboard and Python controllers, and naive straight-line movement that could intersect racks. We therefore addressed control reliability before treating navigation as complete.

3.2 Fixing wheel actuation and control ownership

We configured commands for all four driven wheels and established a clearer distinction between manual and autonomous control. The autonomous controller could issue coordinated wheel speeds without competing with the keyboard graph. We also worked on restoring manual control when needed. These changes were applied in the simulation workflow while keeping the base warehouse assets intact.

3.3 Four-wheel skid-steer control

For the original robot, we used skid-steer/differential-drive-style wheel commands: the wheels on each side received coordinated speeds. The controller used robot pose (x, y and heading), distance to the next waypoint, and heading error to compute forward and turning commands. Speed limits, braking and deliberate corner alignment reduced overshoot and unstable turning.

3.4 A* path planning using static warehouse geometry

Directly steering toward a target is insufficient when racks block the line of travel. We developed an A*-based planner that used the known static USD collision geometry to identify occupied regions, apply obstacle clearance and find a traversable path. The route was converted into waypoints for the motion controller. This was obstacle-aware planning against known geometry, not real-time detection of new obstacles.

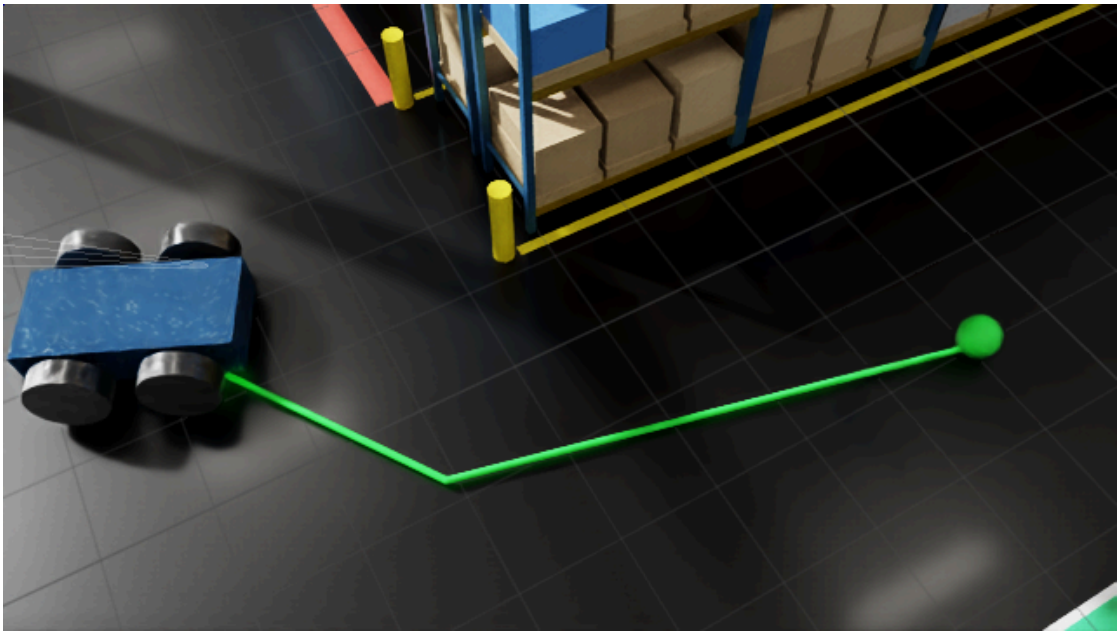


Figure — Actual Isaac Sim evidence of an obstacle detour in the original robot implementation.

3.5 Verification and recorded results

Our second progress presentation documents a successful original-robot run with two of two waypoints reached, a final goal error of 0.231 m and 12 of 12 passing unit tests. It also includes measured motion/trajectory evidence and distinguishes actual simulation results from illustrative diagrams. These metrics apply only to the original IsaacBot implementation, not to the later Carter robot.

Recorded metric	Result	Applies to
Waypoint completion	2 / 2	Original IsaacBot
Final goal error	0.231 m	Original IsaacBot
Unit tests	12 / 12 passed	Original planner/controller

Recorded prototype validation — Isaac Sim 5.1

Measurement	Result
Result	complete
Waypoints reached	2 / 2
Simulation duration	25.85 s
Final goal error	0.231 m
Arrival tolerance	0.25 m
Final measured speed	0.000113 m/s
Controller and planner tests	12 passed

Generated from recorded test results; not a screenshot of the simulator UI.

Figure — Original robot's recorded navigation-validation results.

Outcome

We progressed from manual movement to a tested original-robot autonomous navigation prototype that could plan around known static obstacles and follow waypoints. We also documented the control and test evidence needed to make the results reproducible.

MobilityGen, Carter, ROS 2 and Final Demonstrations

4.1 Why we changed robot setups

Following the team lead's direction, we explored NVIDIA MobilityGen and transitioned from our original robot to a Carter-based mobile robot setup in the warehouse. We checked robot placement, movement and compatibility with the existing scene. MobilityGen was part of our mobility workflow and robot/environment exploration; we did not claim that MobilityGen itself implemented our custom A* planner.

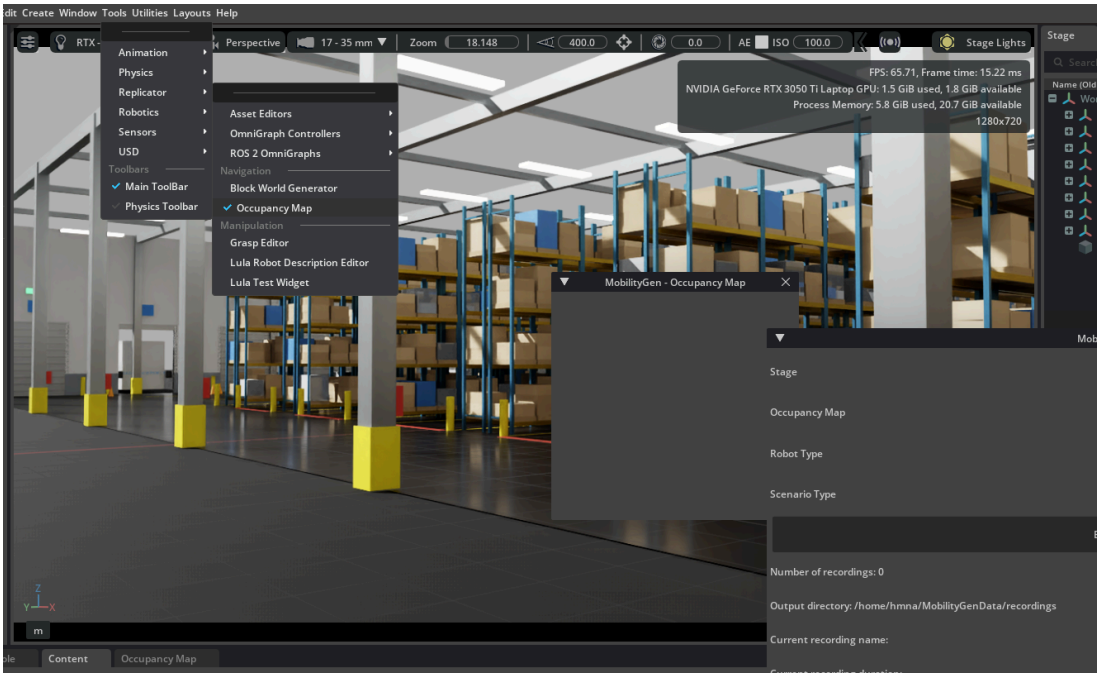


Figure — Carter-based robot setup after the MobilityGen exploration.

4.2 Attempting the official ROS 2 Nav2 workflow

We also followed NVIDIA's official Isaac Sim 5.1.0 ROS 2 Navigation tutorial. In a separate experiment, we worked with a ROS 2 workspace, Isaac Sim ROS bridge, Nav2 and RViz2. We made progress displaying the warehouse occupancy map in RViz2, but the combined

workload repeatedly caused freezes or unresponsive behavior. We also encountered intermittent simulation-clock communication problems. We therefore could not validate complete end-to-end Nav2 goal execution in that experimental setup.

Official reference:
https://docs.isaacsim.omniverse.nvidia.com/5.1.0/ros2_tutorials/tutorial_ros2_navigation.html

4.3 Choosing a directly integrated Python navigation system

To complete the core Control & Navigation demonstration, we implemented a custom Python-based workflow operating with the Carter robot inside Isaac Sim. It used a known warehouse occupancy map and simulator-provided robot pose. A* generated a route through navigable cells, and the controller issued linear and angular velocity commands to follow it. The custom workflow did not require the ROS 2 Nav2 stack to execute the mission.

Stage	Input	Output
Goal selection	Selected destination or waypoint queue	Target position
Map-based planning	Known occupancy map and robot pose	Planned route
Motion control	Route, pose and heading error	Linear and angular velocity
Feedback	Simulator pose and progress	Navigation status and display

4.4 Custom navigation interface

We created a Warehouse Control & Navigation panel with goal selection, add waypoint, run queue, save/load mission, pause, resume, cancel, emergency stop and manual driving controls. It displayed the map, robot marker, planned green route, pose, velocity commands and navigation status. The interface allowed a repeatable demonstration without requiring manual steering for every step.

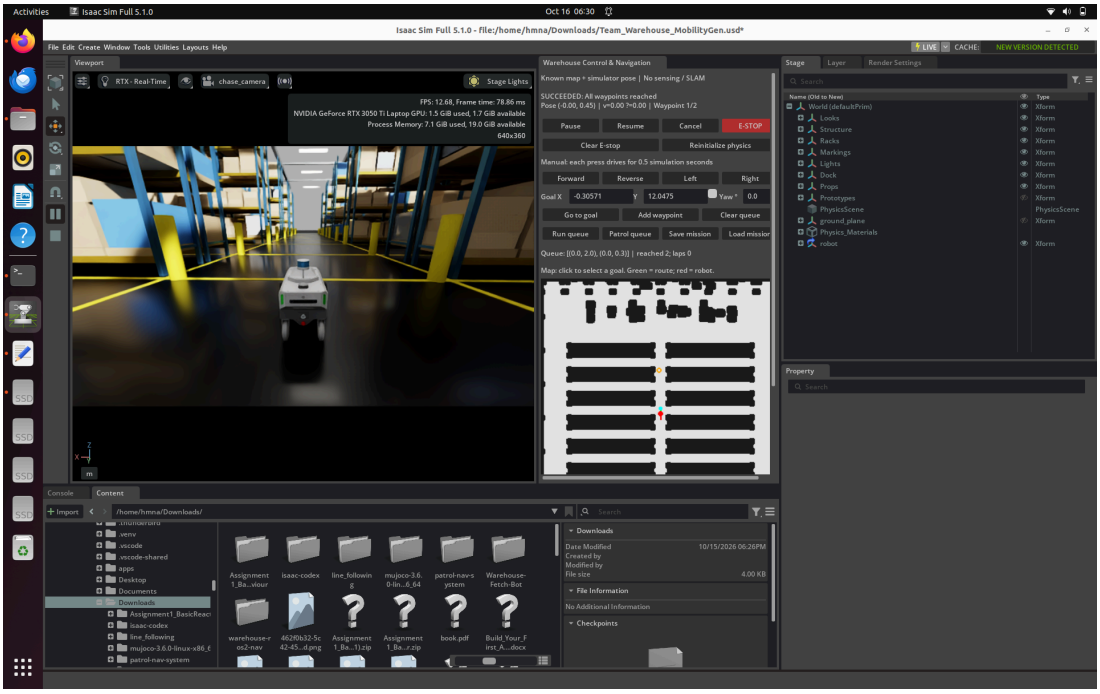


Figure — Custom Carter navigation interface and map feedback.

4.5 Experiment A: two-waypoint mission

We loaded and executed a two-waypoint queue. The robot first adjusted its heading, then moved along the route and reached the requested waypoints. The panel reported “SUCCEEDED: All waypoints reached.” This verified goal/waypoint execution in the Carter-based custom navigation implementation.

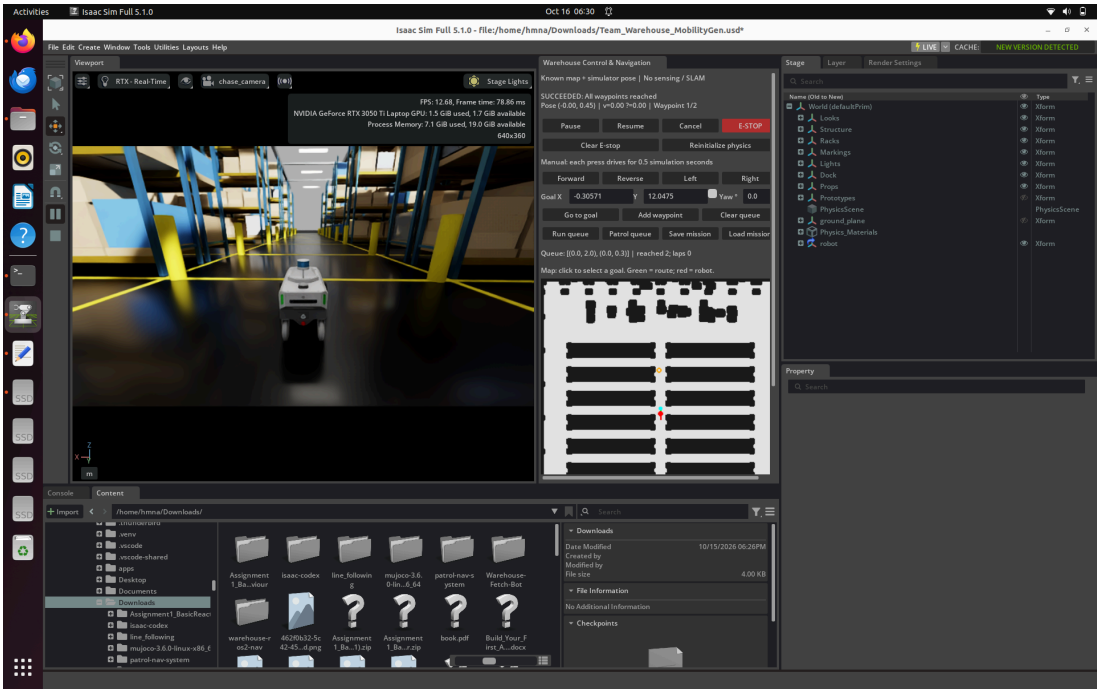


Figure — Completed two-waypoint mission with success status.

4.6 Experiment B: longer-distance goal

After the short mission, we selected a farther goal along the aisle, with target Y = 8.0 m. The robot travelled toward that location and the panel reported success near the destination. This demonstrated a longer continuous navigation run than the initial short-distance checks.

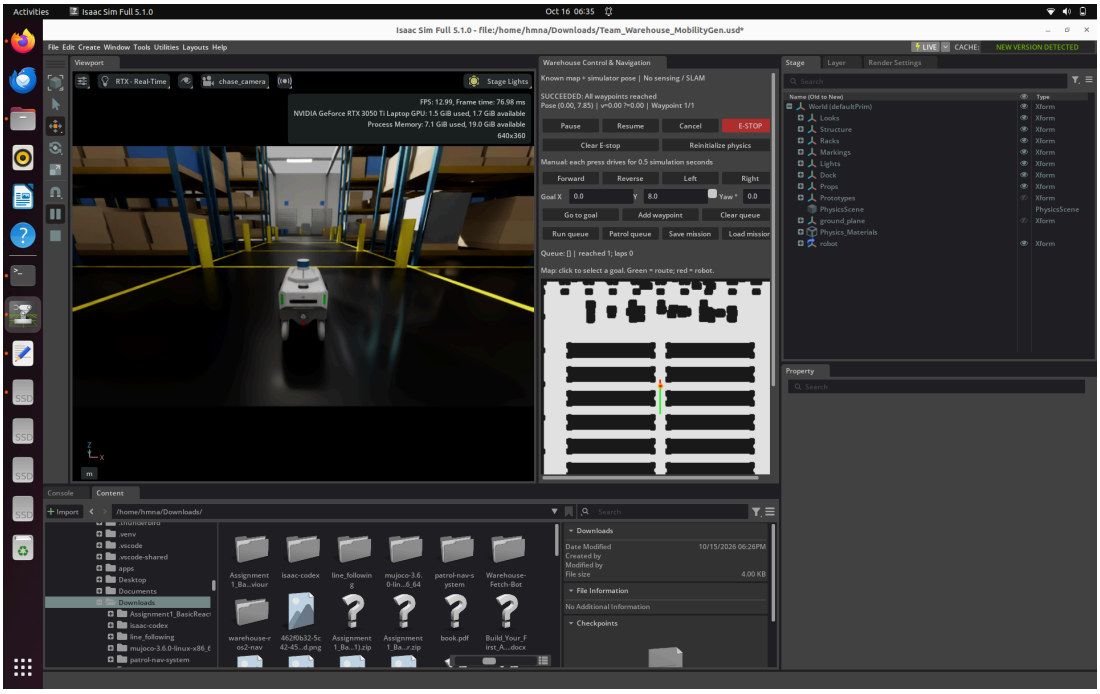


Figure — Longer-distance navigation test in the warehouse aisle.

4.7 Experiment C: goal between racks

We selected a goal between storage racks. The displayed planned route included a turn and passed through an available aisle, and the robot was observed moving along it. The available screenshot captures the robot during navigation; it does not independently show final arrival for this specific run. We therefore describe it as a demonstrated route-following observation, not a separately verified completion.

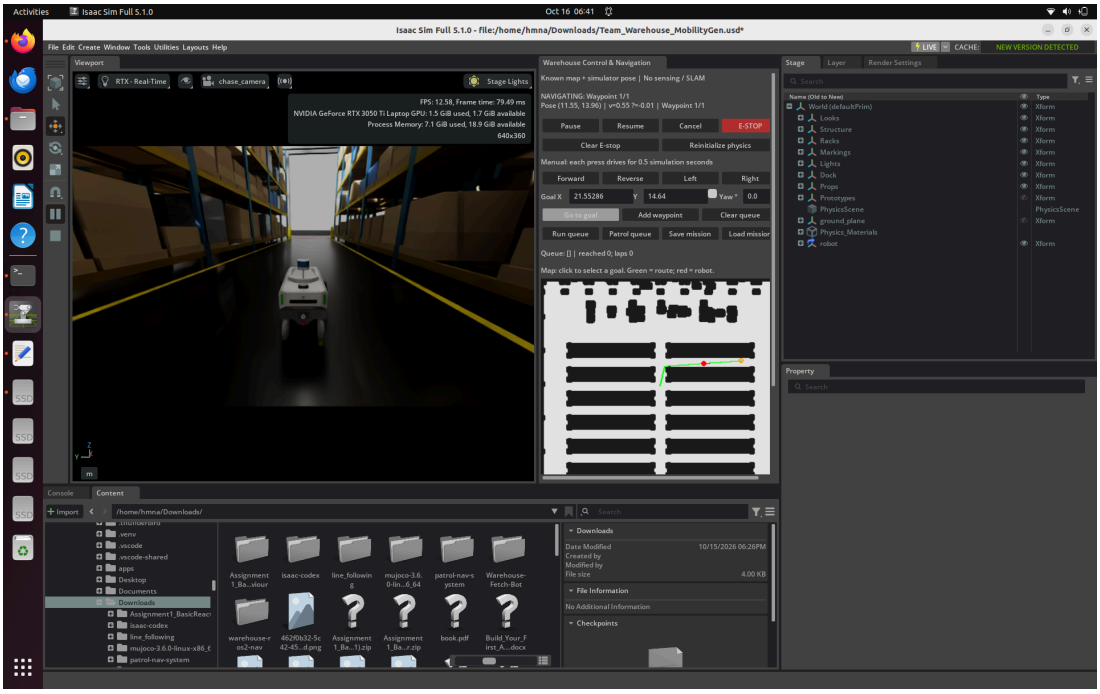


Figure — Planned route and robot motion between warehouse racks.

4.8 Final ROS 2 sample breakthrough

Near the final presentation, we successfully launched NVIDIA's official ROS 2 warehouse navigation sample in Isaac Sim 5.1.0. The screenshot shows Nova Carter in the sample warehouse, RViz2 displaying an occupancy map and LiDAR visualization, and a camera image window. This is a meaningful integration and visualization milestone after the earlier difficulties. The screenshot alone does not demonstrate that a Nav2 goal was sent and completed.

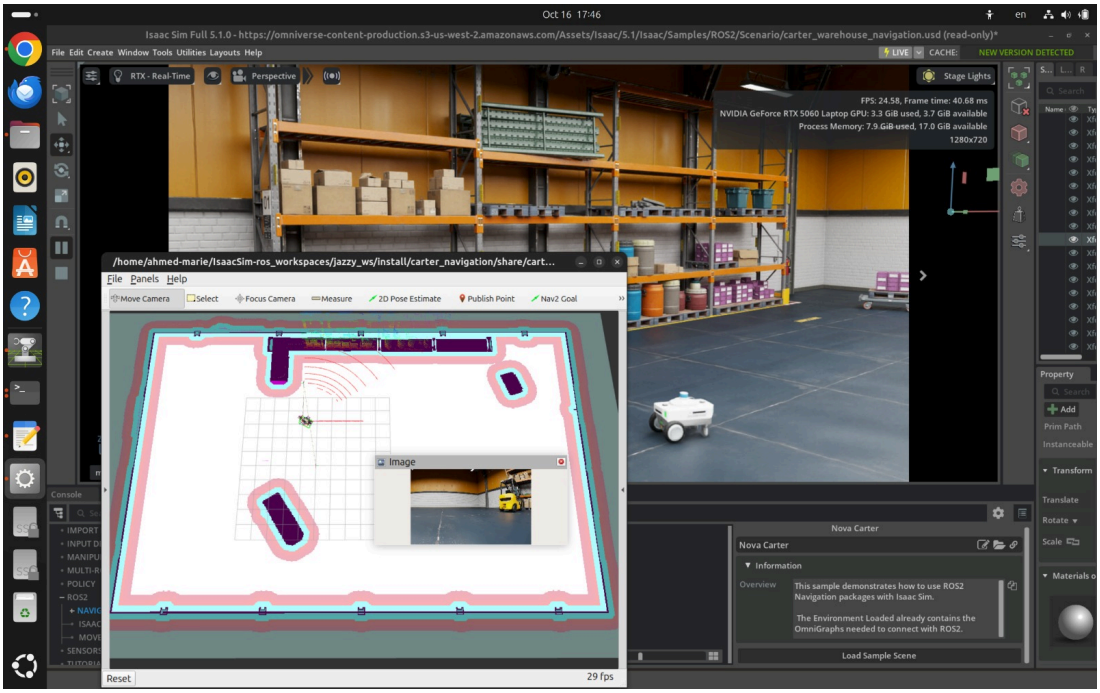


Figure — Official ROS 2 sample running with RViz2 map, LiDAR and camera visualization.

Outcome

We delivered a working Carter-based custom map navigation demonstration, documented the limitations of the earlier Nav2 integration, and showed the official ROS 2 sample running with visualization. The custom Carter implementation and the official ROS 2 sample are separate demonstrations and should not be presented as one merged navigation stack.

Overall Results and Evidence Boundaries

Capability	Evidence status
Manual four-wheel robot movement	Demonstrated
Warehouse integration	Demonstrated
Camera and LiDAR visualization	Demonstrated; not autonomous sensing
Original IsaacBot A* navigation	Tested; 2/2 waypoints, 0.231 m error, 12/12 unit tests
Carter custom map navigation	Tested; two-waypoint success and longer goal success
Route between warehouse racks	Observed while navigating; arrival not shown in supplied frame
ROS 2/Nav2 custom-scene integration	Map display achieved; end-to-end goal not validated
Official ROS 2 sample	Isaac Sim + RViz2 + LiDAR/camera visualization demonstrated
Live SLAM, dynamic avoidance, VLA integration	Not validated

Challenges, Engineering Decisions and Lessons Learned

This training involved several linked engineering challenges. The storage and VRAM constraints forced us to consider the simulator's hardware requirements before development could begin. The first robot taught us that keyboard movement does not automatically imply reliable autonomous wheel control. Camera and LiDAR displays taught us to distinguish sensor visualization from using sensor data for navigation. Our A* experiments showed why warehouse geometry and obstacle clearance matter. The ROS 2 trials highlighted synchronization and performance costs when combining multiple robotics tools. Finally, the custom Carter navigation workflow showed that we could demonstrate planning and feedback control inside Isaac Sim while continuing ROS 2 exploration separately.

Deliverables and Reproducibility

Our public project repository is https://github.com/Hamna12/warehouse-control-navigation. It contains the navigation code and project files documented in the presentation, including the warehouse USD scene, occupancy-map data and README. The custom project uses local

file paths and an Isaac Sim connection service, so a new user may need to configure the scene paths, Python environment and simulator connection to reproduce it.

Conclusion

Our Control & Navigation work progressed from an initially blocked Isaac Sim installation to a working manually driven robot, warehouse integration, sensor visualization, Python tooling, tested autonomous A* navigation on the original robot, and a later Carter-based map navigation demonstration. We also explored the official ROS 2/Nav2 approach, documented its integration difficulties and eventually launched the official ROS 2 sample with RViz2 visualization. The main contribution is a traceable engineering progression from basic movement to demonstrated map-based autonomy, with clear separation between implemented functions, measured results and features not yet validated.

References

NVIDIA Isaac Sim 5.1.0 ROS 2 Navigation tutorial: https://docs.isaacsim.omniverse.nvidia.com/5.1.0/ros2_tutorials/tutorial_ros2_navigation.html

https://husarion.com/tutorials/ros2-tutorials/ros2/

https://ocw.tudelft.nl/courses/hello-real-world-ros-robot-operating-system/

https://www.edx.org/learn/robotics/delft-university-of-technology-hello-real-world-with-ros-robot-operating-system

https://www.ros2lab.com/

https://github.com/isaac-sim/IsaacSim

https://docs.isaacsim.omniverse.nvidia.com/6.0.1/

https://blogs.nvidia.com/blog/mega-omniverse-blueprint/